

Object Oriented Programming for Data Processing

23 June 2025

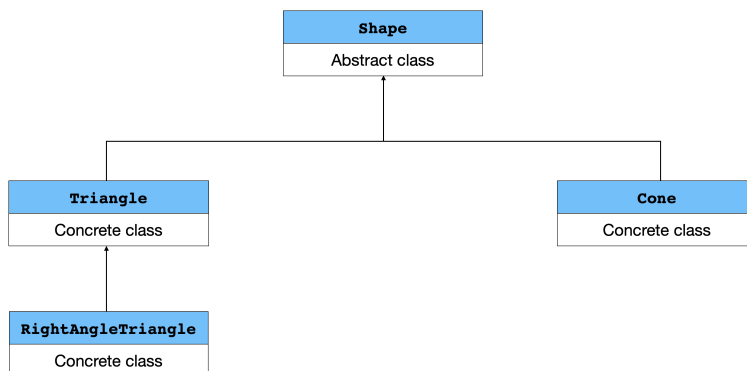
Your exam material (code files, plots, datafiles, etc.) must be submitted via google classroom by 13:00 as a single zip file.

C++ evaluation will be based on: correct syntax, proper return types and arguments of functions, data members and class interfaces, setters/getters, correct mathematical expressions, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, comments throughout the notebook/scripts, labels, legends and plot styling and clarity in general.

Part 1 – C++

Implement the following classes and inheritance hierarchy and submit also a file `app.cpp` to showcase the implementation.



- The abstract class **Shape** represents 2D and 3D geometric shapes in general. **Shape** must have the string attribute **label** and it must require any concrete class inheriting from it to specify a **print** method.
- The concrete class **Triangle** must have a regular constructor that takes the three side lengths as input; this class must include a **Perimeter** method and it must determine and store the three angles. [As a reminder, the angle γ adjacent to the sides a and b and opposite the side c is given by $\cos \gamma = (a^2 + b^2 - c^2)/(2ab)$.]
- In the case of the concrete class **RightAngleTriangle**, it is enough to specify two sides thanks to Pythagora's theorem. Use the named constructor idiom so that the user can specify either the lengths of the hypotenuse and a leg, or of the two legs.
- Design the concrete class **Cone**; this includes calculating and storing the surface area and volume of the cone instance.

Part 2 – Python

Use a Python notebook or Python scripts to simulate an epidemic among N individuals who live in an $L \times L$ 2-dimensional region with periodic boundaries conditions.

1. Place the individuals randomly in the region: all of them are healthy with the exception of the one labeled by $i = 0$ (patient zero).
2. At each timestep, healthy individuals move around by traveling a fix distance ℓ in random direction: update their positions remembering to enforce periodic boundary conditions.
3. Following the update of all positions, check over all individuals to determine how their health states will need to be updated:
 - a sick individual dies with probability p_d and recovers with probability p_h (with $p_d + p_h < 1$), i.e., extract a random number x from a uniform distribution between 0 and 1 and compare it to p_d and $p_d + p_h$ to determine whether the individual dies ($0 \leq x < p_d$), recovers ($p_d \leq x < p_d + p_h$) or remains sick ($p_d + p_h \leq x < 1$);
 - healthy individuals can become sick if they interact with other sick individuals; to model this, every time you update the positions of the individuals, you must compute the $N \times (N - 1)/2$ distances among them; if the individual under consideration is within a distance d from n sick individuals, extract n random numbers from a uniform distribution between 0 and 1 and the individual will turn sick if one or more of them are smaller than p_s ;
 - once you know what will happen to each individual, apply the changes to the whole population making sure you remove individuals killed by the epidemic.
4. Display the evolution over time of the number of healthy individuals, of the number of sick individuals, and of the total number of individuals in a single plot.

N , L , p_d , p_h , p_s , and d are specified by the user at the start of the simulation.